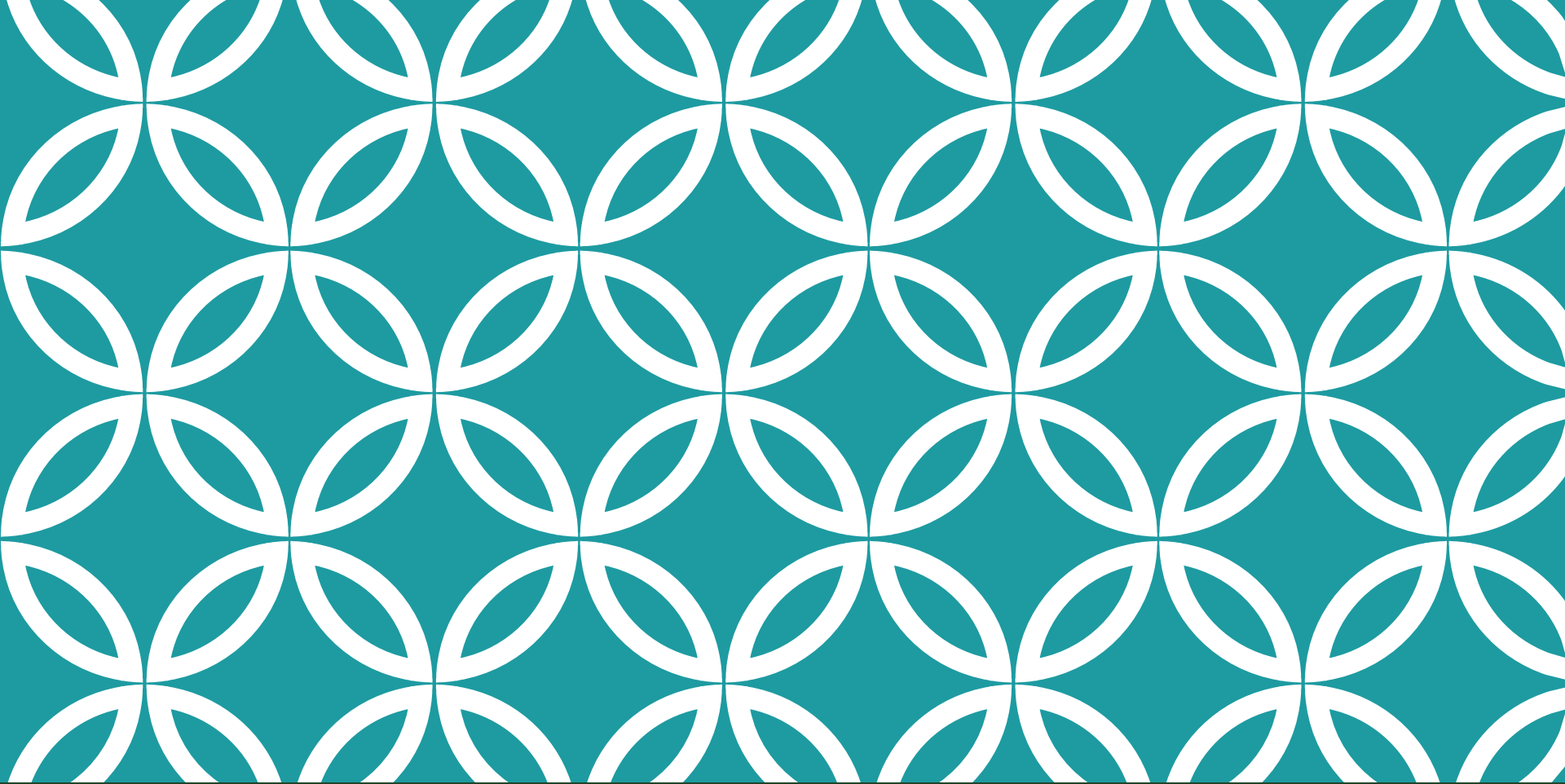




8086 MICROPROCESSOR

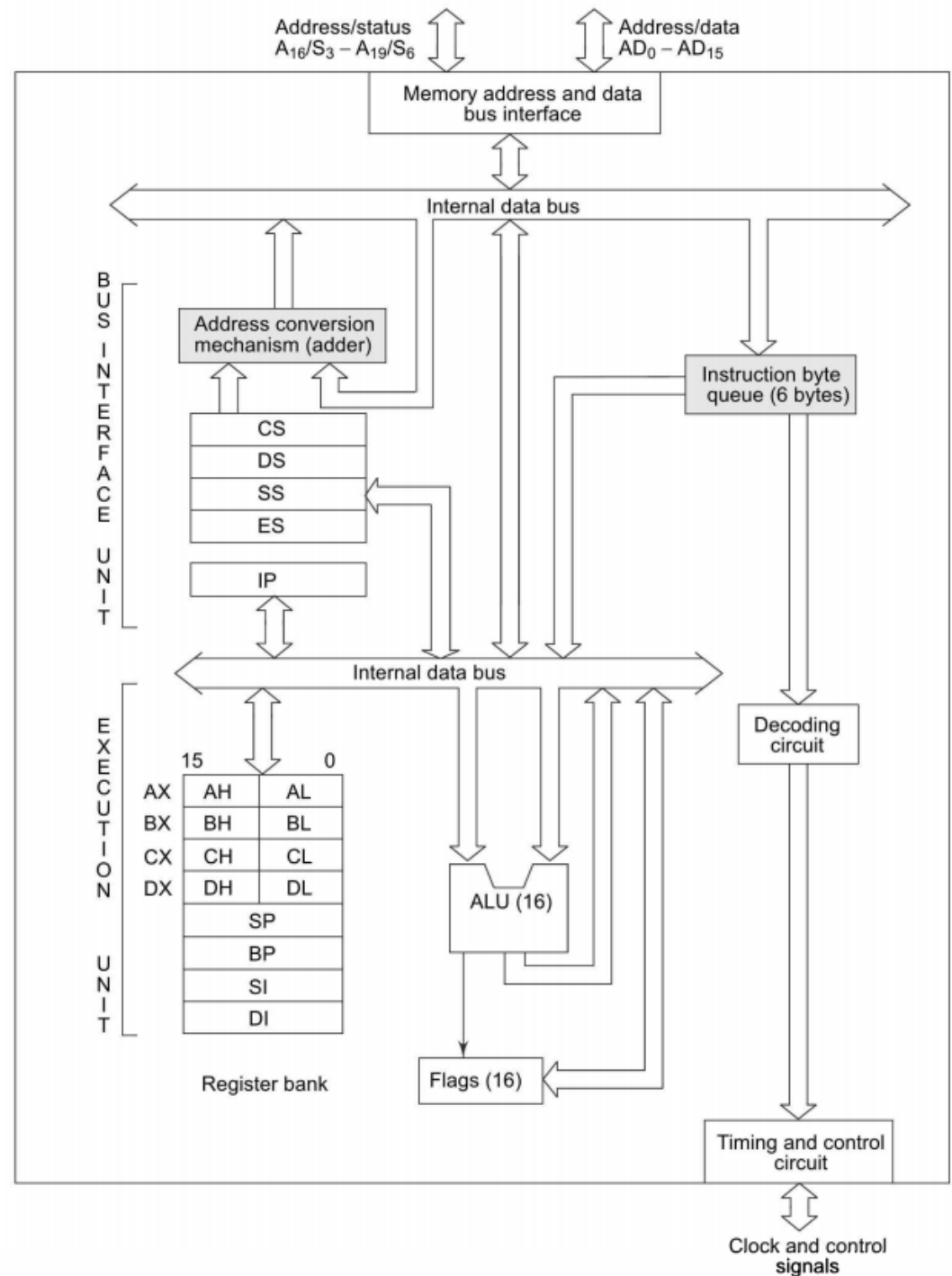
**By Dr. Veerapu
Goutham**

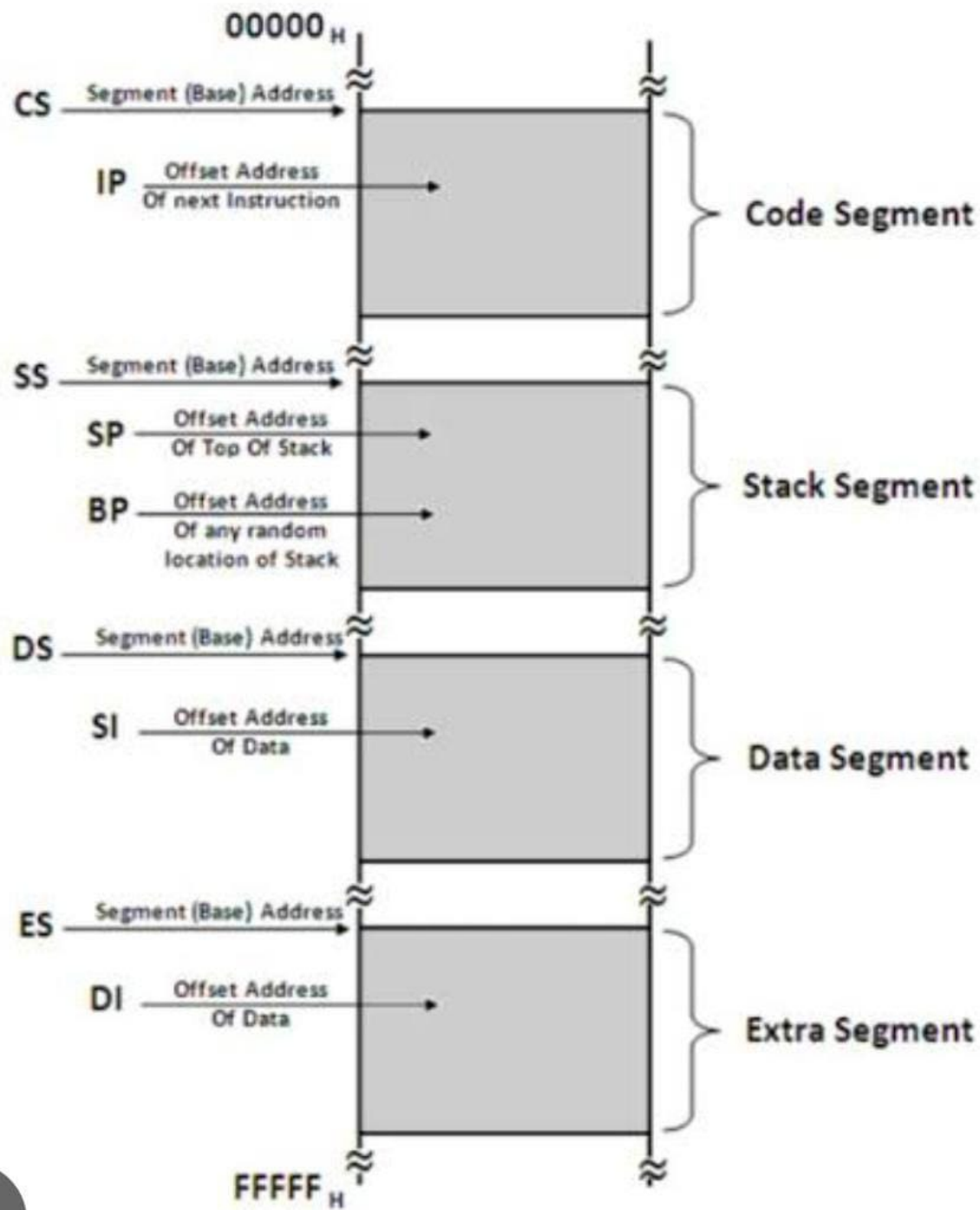


Contents:

1. 8086 Architecture
2. Flag register

ARCHITECTURE OF 8086





Intel 8086 : Memory segmentation (Organization)

- 8086 can access maximum of 1MB of memory with its 20-bit address bus.
 - $2^{20}=1\text{MB}$.
- This 1MB of physical memory space of 8086 is divided/segmented in to 16-logical segments of 64kB each.
 - $64\text{kB} \times 16 = 1\text{MB}$ (as shown in the figure)
- These 16 logical segments of 64kB each are further grouped in to 4-segments namely
 - Code segment** : Reserved for storing program code.
 - Data segment** : Reserved for storing user data
 - Extra (alternate data) segment** : This is also Reserved for storing User data
 - Stack segment** : Reserved for temporary data storage by processor (stack area).

Code segment		Data segment	
Logical segment1 64kB	Logical segment2 64kB	Logical segment3 64kB	Logical segment4 64kB
Logical segment5 64kB	Logical segment6 64kB	Logical segment7 64kB	Logical segment8 64kB
Logical segment9 64kB	Logical segment10 64kB	Logical segment11 64kB	Logical segment12 64kB
Logical segment13 64kB	Logical segment14 64kB	Logical segment15 64kB	Logical segment16 64kB
Extra segment		Stack segment	

Intel 8086 : Memory segmentation (Organization)

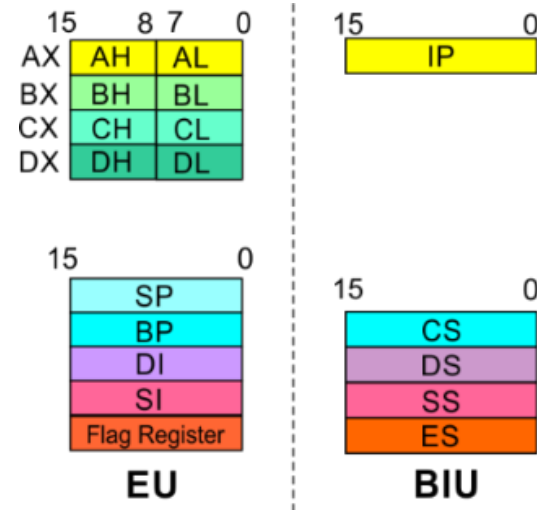
Note: For memory segmentation,

- There must be 4-groups.
- All the 4-groups must have at least 1-logical segment for each.
- Any logical segment and any number of logical segments can be assigned to each group.

Code segment		Data segment	
Logical segment1 64kB	Logical segment2 64kB	Logical segment3 64kB	Logical segment4 64kB
Logical segment5 64kB	Logical segment6 64kB	Logical segment7 64kB	Logical segment8 64kB
Logical segment9 64kB	Logical segment10 64kB	Logical segment11 64kB	Logical segment12 64kB
Logical segment13 64kB	Logical segment14 64kB	Logical segment15 64kB	Logical segment16 64kB
Extra segment		Stack segment	

MEMORY ACCESS

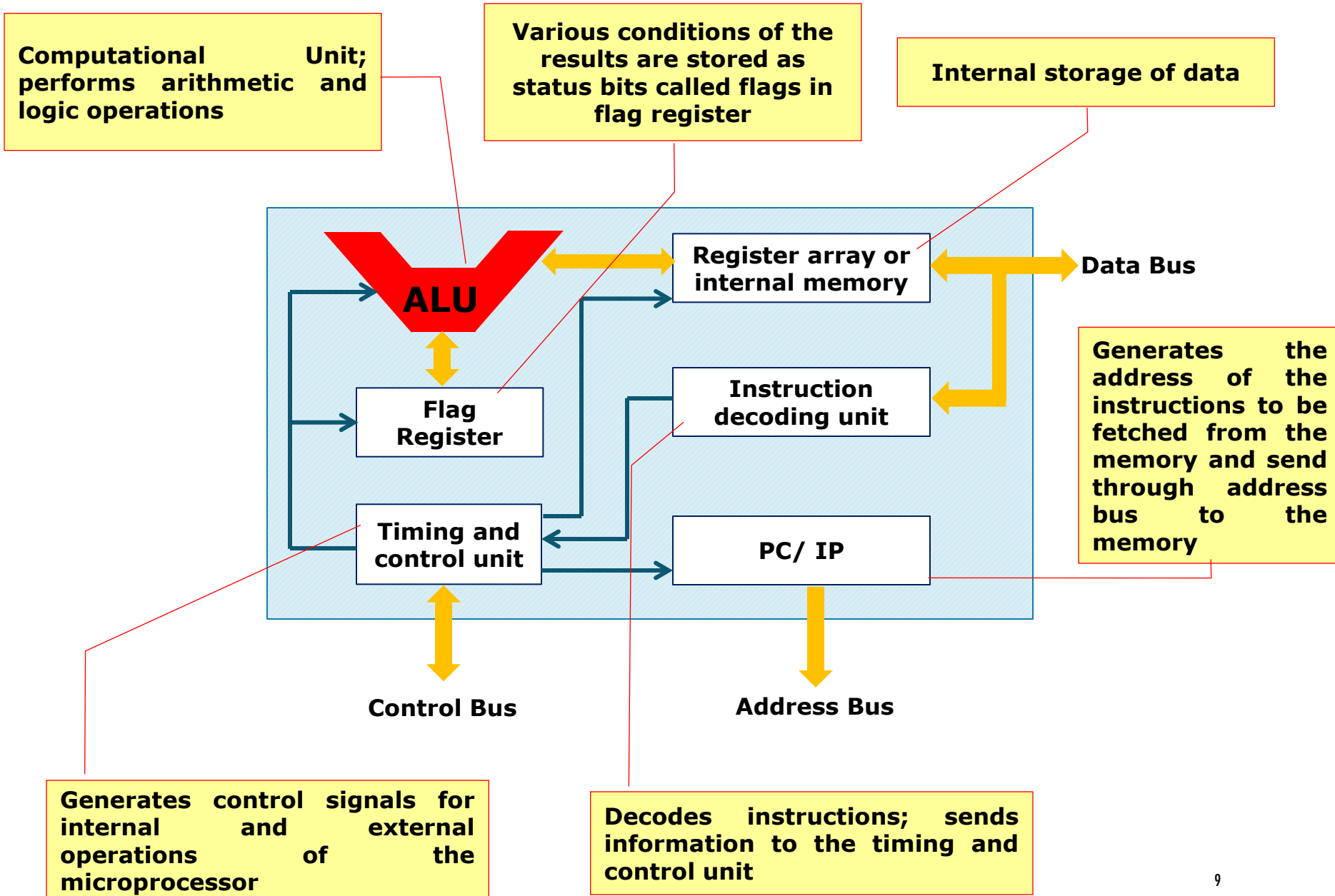
- 20 Address lines $\Rightarrow$ 8086 can address up to $2^{20} = 1\text{M}$ bytes of memory
- However, the largest register is only 16 bits
- Physical Address will have to be calculated
Physical Address : Actual address of a byte in memory. i.e. the value which goes out onto the address bus.
- Memory Address represented in the form –
Seg : Offset (Eg - 89AB:F012)
- Each time the processor wants to access memory, it takes the contents of a segment register, shifts it one hexadecimal place to the left (same as multiplying by 16_{10}), then add the required offset to form the 20- bit address



16 bytes of contiguous memory

89AB : F012 $\rightarrow$ 89AB $\rightarrow$ 89AB0 (Paragraph to byte $\rightarrow 89AB \times 10 = 89AB0$)
F012 $\rightarrow$ 0F012 (Offset is already in byte unit)
+ -----
98AC2 (The absolute address)

MICROPROCESSOR FUNCTIONAL BLOCKS



8086 Microprocessor

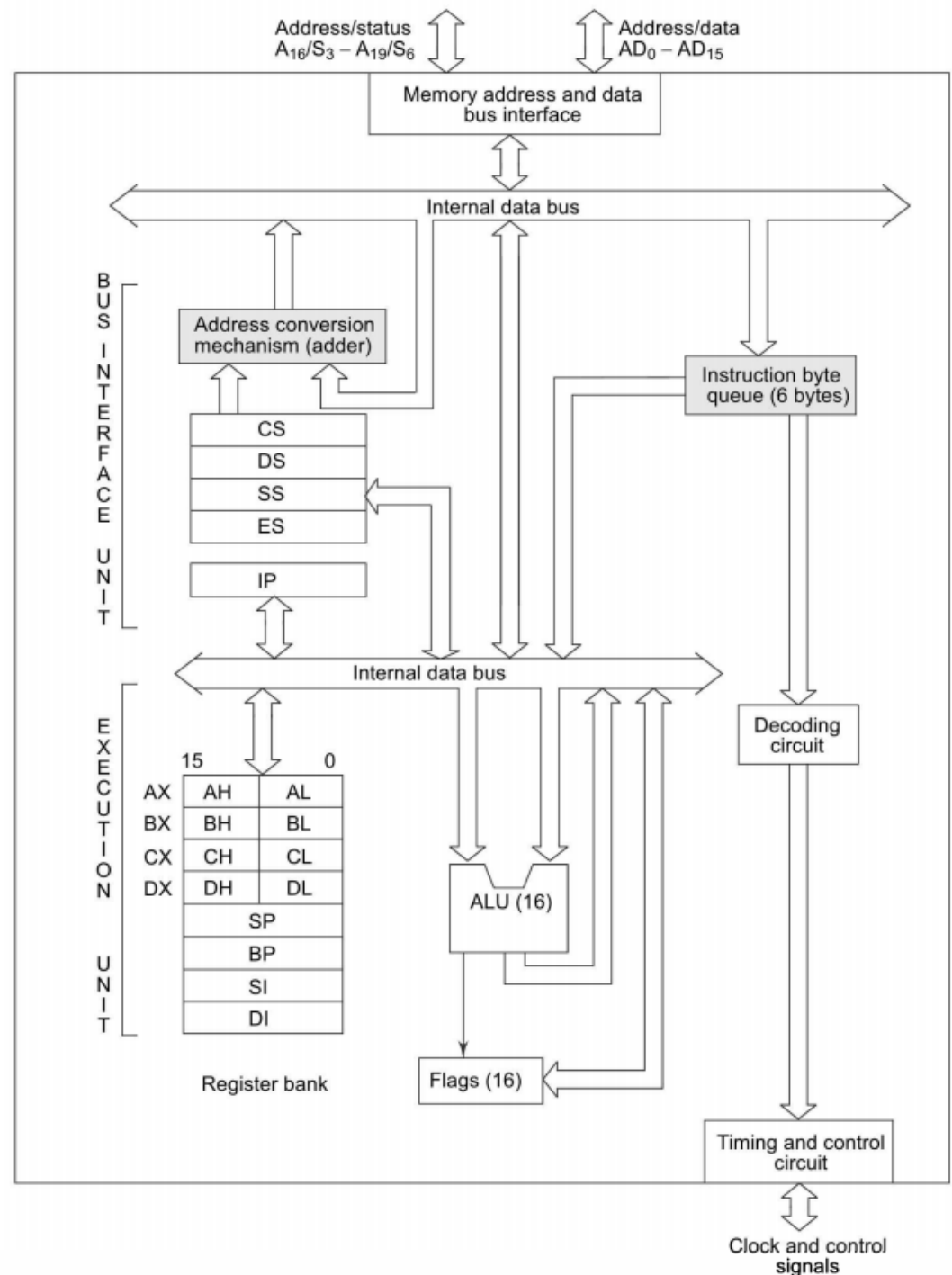
Bus Interface Unit (BIU)

BIU fetches instructions, reads data from memory and I/O ports, writes data to memory and I/O ports.

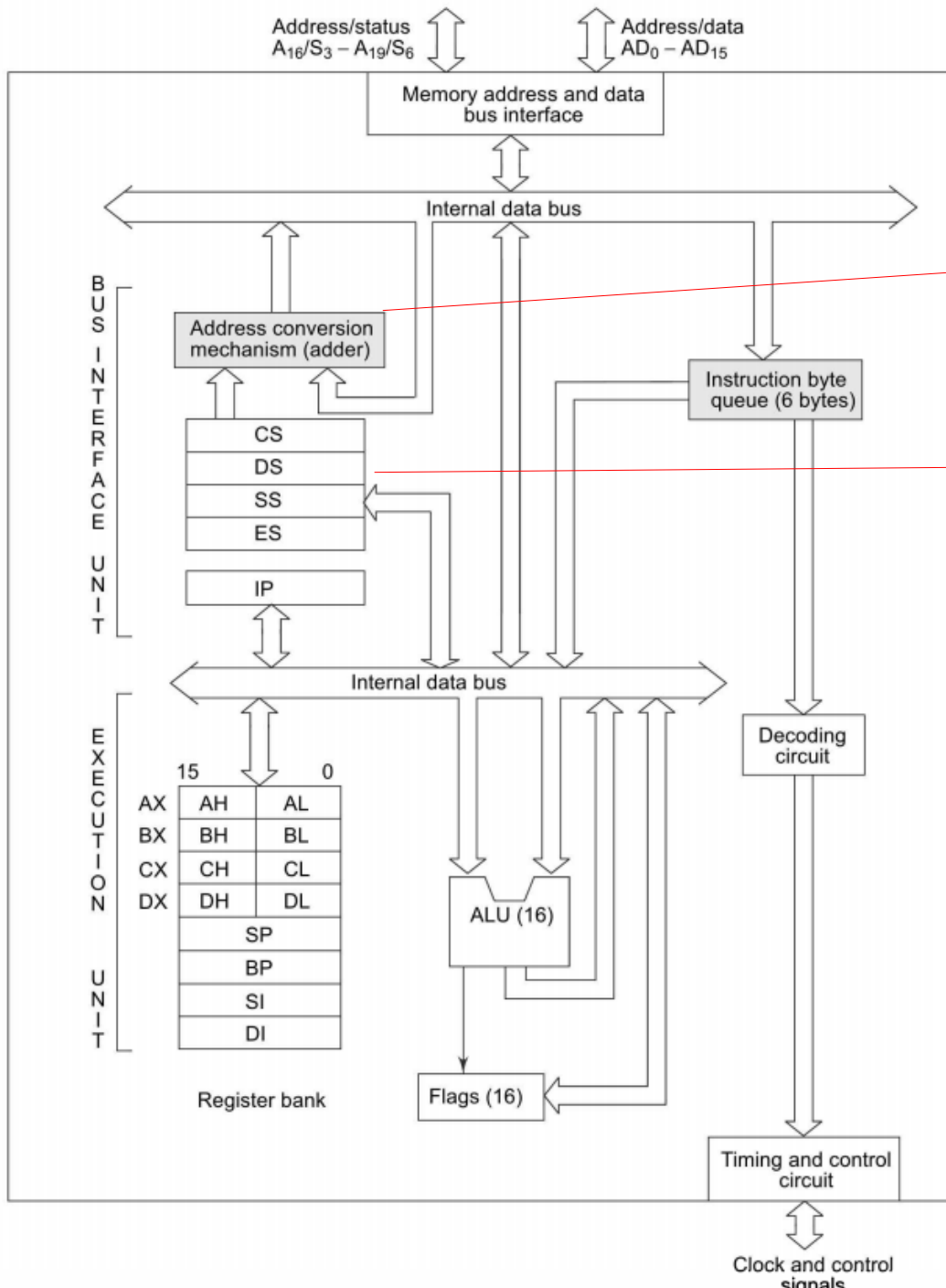
Execution Unit (EU)

EU executes instructions that have already been fetched by the BIU.

BIU and EU functions separately.



Bus Interface Unit (BIU)

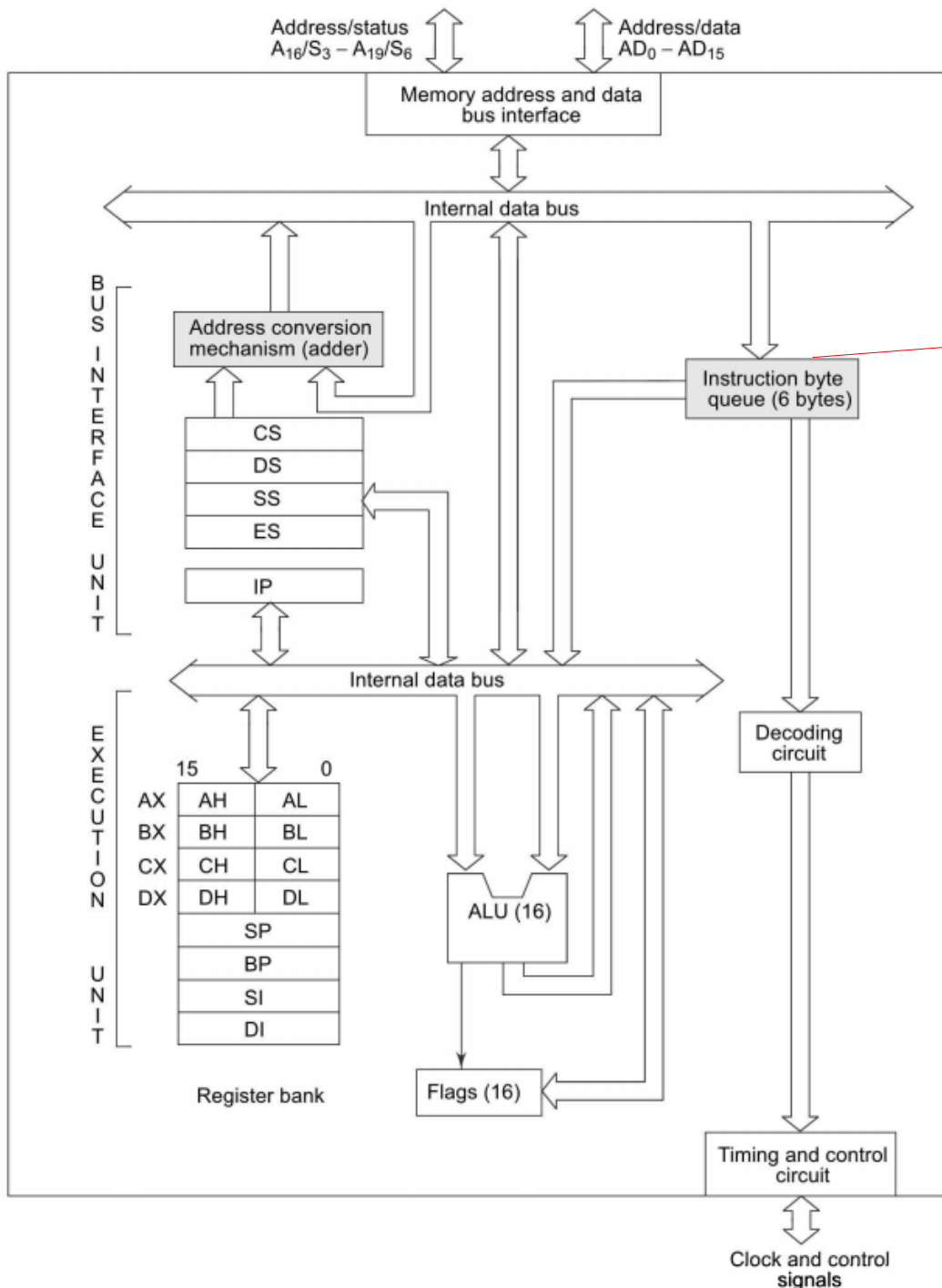


Dedicated Adder to generate 20 bit address

Four 16-bit segment registers

**Code Segment (CS)
Data Segment (DS)
Stack Segment (SS)
Extra Segment (ES)**

Bus Interface Unit (BIU)

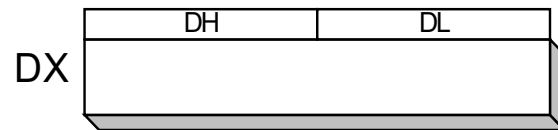
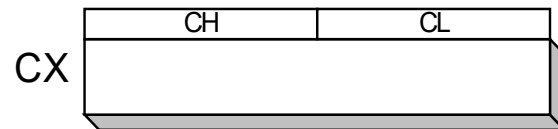
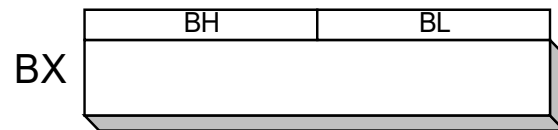
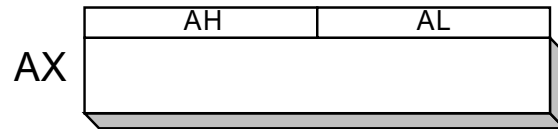


Instruction queue

- A group of First-In-First-Out (FIFO) in which up to 6 bytes of instruction code are pre fetched from the memory ahead of time.
- This is done in order to speed up the execution by overlapping instruction fetch with execution.
- This mechanism is known as pipelining.

8086 REGISTERS

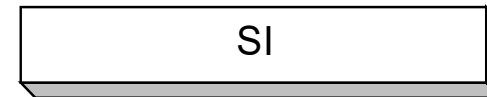
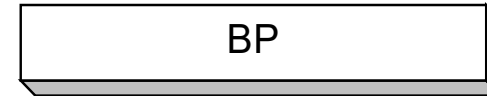
General Purpose



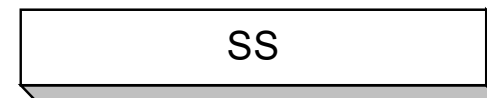
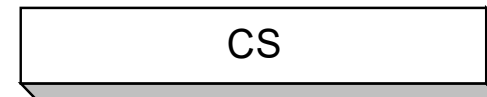
Status and Control



Index



Segment



GENERAL PURPOSE REGISTERS

15	H	8	7	L	0
AX (Accumulator)					
AH			AL		
BX (Base Register)					
BH			BL		
CX (Used as a counter)					
CH			CL		
DX (Used to point to data in I/O operations)					
DH			DL		

AX - the Accumulator
BX - the Base Register
CX - the Count Register
DX - the Data Register

Normally used for storing temporary results

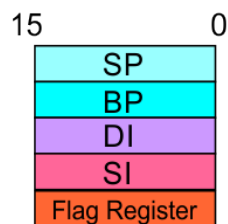
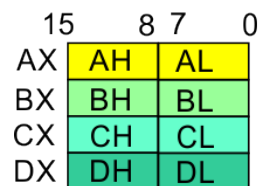
Each of the registers is 16 bits wide (**AX, BX, CX, DX**)

Can be accessed as either 16 or 8 bits AX, AH, AL

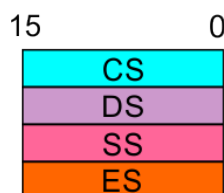
EU Registers

Accumulator Register (AX)

- Consists of two 8-bit registers AL and AH, which can be combined together and used as a 16-bit register AX.
- AL in this case contains the low order byte of the word, and AH contains the high-order byte.
- The I/O instructions use the AX or AL for inputting / outputting 16 or 8 bit data to or from an I/O port.
- Multiplication and Division instructions also use the AX or AL.



EU

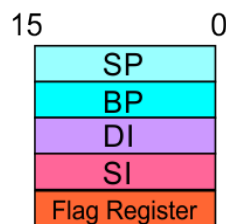
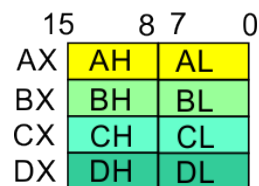


BIU

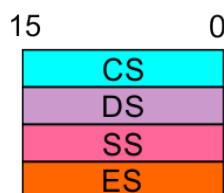
EU Registers

Base Register (BX)

- Consists of two 8-bit registers BL and BH, which can be combined together and used as a 16-bit register BX.
- BL in this case contains the low-order byte of the word, and BH contains the high-order byte.
- This is the only general purpose register whose contents can be used for addressing the 8086 memory.
- All memory references utilizing this register content for addressing use DS as the default segment register.



EU



BIU

EU Registers

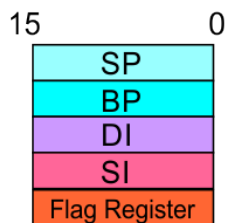
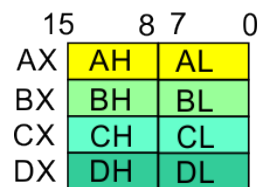
Counter Register (CX)

- Consists of two 8-bit registers CL and CH, which can be combined together and used as a 16-bit register CX.
- When combined, CL register contains the low order byte of the word, and CH contains the high-order byte.
- Instructions such as **SHIFT**, **ROTATE** and **LOOP** use the contents of CX as a counter.

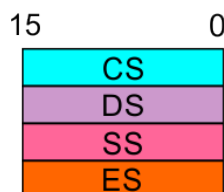
Example:

The instruction **LOOP START** automatically decrements CX by 1 without affecting flags and will check if [CX] = 0.

If it is zero, 8086 executes the next instruction; otherwise the 8086 branches to the label START.



EU

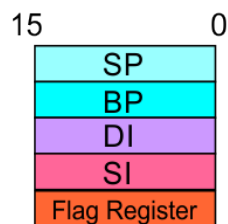
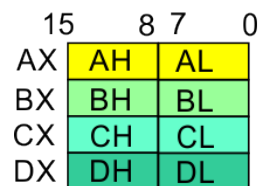


BIU

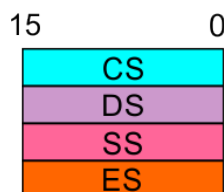
EU Registers

Data Register (DX)

- Consists of two 8-bit registers DL and DH, which can be combined together and used as a 16-bit register DX.
- When combined, DL register contains the low order byte of the word, and DH contains the high-order byte.
- Used to hold the high 16-bit result (data) in 16 X 16 multiplication or the high 16-bit dividend (data) before a $32 \div 16$ division and the 16-bit remainder after division.



EU



BIU

Pointer and Index Registers

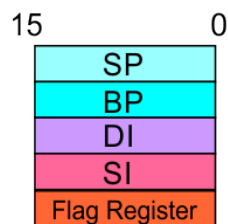
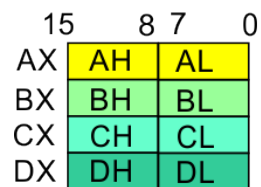
SP	Stack Pointer
BP	Base Pointer
SI	Source Index
DI	Destination Index
IP	Instruction Pointer

- All 16 bits wide, L/H bytes are not accessible
- Used as memory pointers
 - Example: MOV AH, [SI]
 - *Move the byte stored in memory location whose address is contained in register SI to register AH*
- IP is not under direct control of the programmer

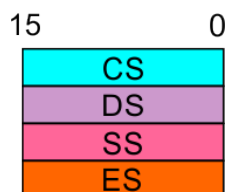
EU Registers

Stack Pointer (SP) and Base Pointer (BP)

- SP and BP are used to access data in the stack segment.
- SP is used as an offset from the current SS during execution of instructions that involve the stack segment in the external memory.
- SP contents are automatically updated (incremented/decremented) due to execution of a POP or PUSH instruction.
- BP contains an offset address in the current SS, which is used by instructions utilizing the based addressing mode.



EU

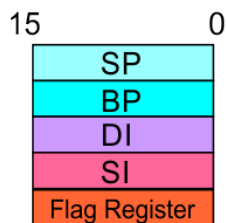
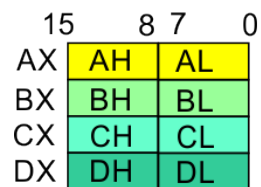


BIU

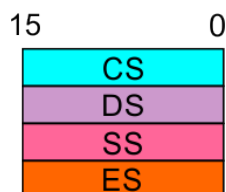
EU Registers

Source Index (SI) and Destination Index (DI)

- Used in indexed addressing.
- Instructions that process data strings use the SI and DI registers together with DS and ES respectively in order to distinguish between the source and destination addresses.



EU



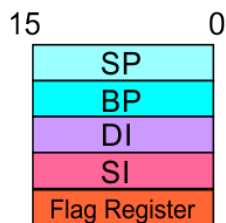
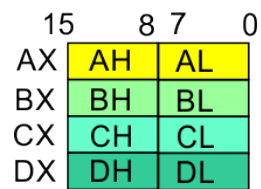
BIU

ARCHITECTURE

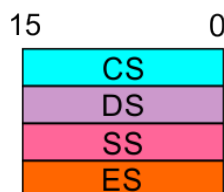
Segment Registers

Instruction Pointer

- 16-bit
- Always points to the next instruction to be executed within the currently executing code segment.
- Its content is automatically incremented as the execution of the next instruction takes place.



EU



BIU

Flag Register

Sign Flag

This flag is set, when the result of any computation is negative

Auxiliary Carry Flag

This is set, if there is a carry from the lowest nibble, i.e, bit three during addition, or borrow for the lowest nibble, i.e, bit three, during subtraction.

Carry Flag

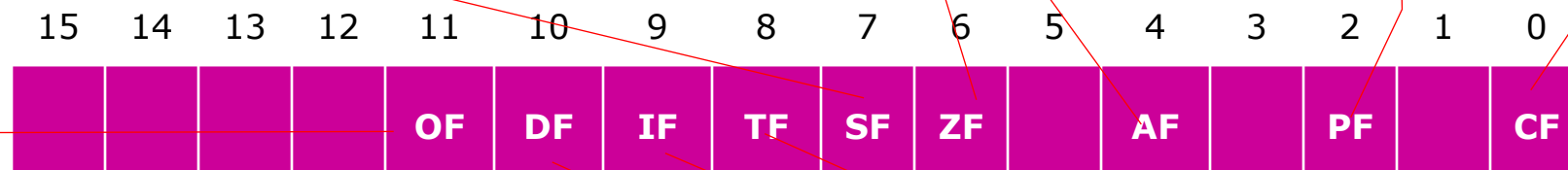
This flag is set, when there is a carry out of MSB in case of addition or a borrow in case of subtraction.

Zero Flag

This flag is set, if the result of the computation or comparison performed by an instruction is zero

Parity Flag

This flag is set to 1, if the result contains even number of 1's ; for odd number of 1's set to zero.



Over flow Flag

This flag is set, if an overflow occurs, i.e, if the result of a signed operation is large enough to accommodate in a destination register. The result is of more than 7-bits in size in case of 8-bit signed operation and more than 15-bits in size in case of 16-bit sign operations, then the overflow will be set.

Trap Flag

If this flag is set, the processor enters the single step execution mode by generating internal interrupts after the execution of each instruction

Direction Flag

This is used by string manipulation instructions. If this flag bit is '0', the string is processed beginning from the lowest address to the highest address, i.e., auto incrementing mode. Otherwise, the string is processed from the highest address towards the lowest address, i.e., auto decrementing mode.

Interrupt Flag

Causes the 8086 to recognize external mask interrupts; clearing IF disables these interrupts.

Addressing modes of 8086

Some of the widely used addressing modes supported by 8086 are listed bellow.

- **Immediate addressing:** One of the operand is data it self on which operation is going to perform.
Example: MOV AX,0005H
- **Direct addressing:** One of the operand is offset/displacement address of the memory location which can be used to calculate the physical address where data is available. **Physical address (20-bits)= Left Shifted segment address + Offset address.**
Example: MOV AX, [5000H]
- **Register addressing:** Both operands are registers where data is available for operation to perform.
Example: MOV AX,CX
- **Register indirect addressing:** One of the operand is register which carries the offset address of the memory location which can be used to calculate the physical address .
Example: MOV AX,[BX]
- **Indexed addressing:** One of the operand is index register (SI/DI) which carries the offset address of the memory location which can be used to calculate the physical address.
Examples: MOV AX, [SI]
MOV CX, [DI]

Addressing modes of 8086

Some of the widely used addressing modes supported by 8086 are listed bellow.

- **Based Indexed addressing:** The operand of this instruction must have any one index register (SI/DI) and any one base register (BX/BP). Both of these registers carries the offset address components. The resultant offset address is the sum of these two addresses which can be used to calculate the physical address where data is available. **Physical address (20-bits)= Left Shifted segment address + Offset address.**

Examples: MOV AX, [BX] [SI]
MOV [BP] [DI], DX

In above example, Resultant offset address=[BX]+[SI]

- **Relative Based Indexed addressing:** This is similar to Based Indexed addressing mode. The difference is, along with any one index register (SI/DI) and any one base register (BX/BP), the operand part of this instruction must have a relative address (In the below example it is given as 50H or 60H). The resultant offset address is the sum of all three address components (relative address, the contents of base and index registers).

Examples: MOV AX, 50H [BX] [SI]
MOV 60H [BP] [DI], DX

In above example, Resultant offset address=50H+[BX]+[SI]

8086 PROGRAMMER'S MODEL

BIU registers
(20 bit adder)

ES	Extra Segment
CS	Code Segment
SS	Stack Segment
DS	Data Segment
IP	Instruction Pointer

EU registers

AX	AH	AL	Accumulator
BX	BH	BL	Base Register
CX	CH	CL	Count Register
DX	DH	DL	Data Register
	SP		Stack Pointer
	BP		Base Pointer
	SI		Source Index Register
	DI		Destination Index Register
	FLAGS		

Register	Name of the Register	Special Function
AX	16-bit Accumulator	Stores the 16-bit results of arithmetic and logic operations
AL	8-bit Accumulator	Stores the 8-bit results of arithmetic and logic operations
BX	Base register	Used to hold base value in base addressing mode to access memory data
CX	Count Register	Used to hold the count value in SHIFT, ROTATE and LOOP instructions
DX	Data Register	Used to hold data for multiplication and division operations
SP	Stack Pointer	Used to hold the offset address of top stack memory
BP	Base Pointer	Used to hold the base value in base addressing using SS register to access data from stack memory
SI	Source Index	Used to hold index value of source operand (data) for string instructions
DI	Data Index	Used to hold the index value of destination operand (data) for string operations



Thank You